MANIPAL SCHOOL OF INFORMATION SCIENCES

Manipal Academy of Higher Education

Linux File Ownership and Permissions

Lab Assignment

Name: Shreyas S Sanil
Roll Number: 261100690003
Program: M.E. Cyber Security
Subject: Computer and Network Security

Academic Year: 2026

Introduction

This experiment is about understanding Linux file ownership, groups and file permissions. The main purpose is to check how different users get access to a file depending on whether they are the owner, a member of the file group, or neither. The experiment also checks how changing permissions and group membership affects the access of users.

Part A – File Ownership and User Access

1. Create the test file

A test file named `incident_report.txt` was created for the experiment.

```
touch incident_report.txt
```

Some sample content was added to the file.

```
echo "This is a dummy incident report." > incident_report.txt
```

The ownership and permissions of the file were checked using:

```
ls -l incident_report.txt
```

The output shows the file owner, group and permission values.

2. Identify the available operations for each user

The users used in this experiment are:

- Alice
- Bob
- Charlie

Their group membership was checked using:

```
id alice
id bob
id charlie
```

The file ownership was checked using:

```
ls -l incident_report.txt
```

Linux permissions are divided into three categories: owner, group and others. The permission used depends on the relationship between the user and the file.

3. Test Charlie's access

Charlie belongs to the `security` group. The file was kept with the required owner and group for the experiment.

The following command was used to switch to Charlie:

```
su - charlie
```

The current user was checked using:

```
whoami
```

Read operation

The following command was used to test whether Charlie could read the file.

```
cat incident_report.txt
```

Result: Charlie could read the file if the applicable permission included read access.

Modify operation

The following command was used to test write access.

```
echo "Charlie test" >> incident_report.txt
```

Result: The command was successful only when Charlie had write permission through the applicable permission class.

Delete operation

The following command was used to test whether Charlie could delete the file.

```
rm incident_report.txt
```

Result: Delete permission mainly depends on the permissions of the directory containing the file, rather than only the file's own write permission.

4. Record Charlie's results

Operation	**Result for Charlie**
Read incident_report.txt	Allowed / Denied
Modify incident_report.txt	Allowed / Denied
Delete incident_report.txt	Allowed / Denied

Part B – Changing File Permissions

1. Change the file permissions

The file permissions were changed to:

```
-rw-rw----
```

The following command was used:

```
chmod 660 incident_report.txt
```

The permission was checked using:

```
ls -l incident_report.txt
```

The permission value `660` means:

- Owner: read and write
- Group: read and write
- Others: no permission

2. Repeat Charlie's test

Charlie was again used to test access to the file.

```
su - charlie
```

The following commands were used:

```
cat incident_report.txt
```

```
echo "Charlie test after chmod" >> incident_report.txt
```

```
rm incident_report.txt
```

The results were recorded below.

Operation	**Result after chmod 660**
Read	Allowed / Denied
Modify	Allowed / Denied
Delete	Allowed / Denied

3. Why did Charlie's access change?

Charlie belongs to the group associated with the file. When the permissions were changed to `-rw-rw----`, the group was given read and write permissions while other users were given no permissions. Therefore, Charlie's access is decided by the group permission because he is a member of the file's group.

The owner and group of the file did not change. Only the permission bits were changed, so the type of access available to the members of the group changed.

Part C – Removing a User from a Group

1. Remove Bob from the security group

Bob was first checked to confirm his current groups.

```
id bob
```

Bob was then removed from the `security` group.

```
sudo gpasswd -d bob security
```

The membership was checked again using:

```
id bob
```

2. Test Bob's access again

The file ownership and permissions were not changed.

They were checked using:

```
ls -l incident_report.txt
```

The permissions should still show:

```
-rw-rw----
```

Bob's access was then tested using the following commands:

```
cat incident_report.txt
```

```
echo "Bob test" >> incident_report.txt
```

The results were recorded below.

Operation	**Result for Bob**
Read	Allowed / Denied
Modify	Allowed / Denied
Delete	Allowed / Denied

3. Why did Bob's effective permissions change?

Bob's effective permissions changed because he was removed from the `security` group. The file's owner, group and permission bits were not changed. However, Bob was no longer a member of the group that received the group permissions.

Therefore, Linux no longer applies the group permission set to Bob. If he is neither the owner nor a member of the file's group, the permissions for "others" are considered.

This shows that file access depends on both the permission bits and the user's relationship with the file owner and group.